

Design by Contract

If the caller meets the requirements, the routine must deliver its promises.

THE AGREEMENT

EXAMPLE: ADD A CHILD TO A TREE NODE

Precondition	Caller must satisfy it before the call.	<code>new</code> refers to an existing node.
Postcondition	Routine guarantees it on successful return.*	<code>new.parent</code> = current <code>child count</code> = old <code>child count</code> + 1
Class invariant	Class stays consistent at public boundaries.	Each existing child points back to its parent node.

*Given valid entry conditions. Invariants hold after creation and at public operation entry/exit, not every internal step.

A contract specifies responsibilities, not implementation details.

The caller guarantees valid use; the supplier guarantees the agreed result.

What contracts change

Explicit responsibilities make software easier to reuse, debug, and extend.

Locate the bug

Broken precondition: caller bug. Broken postcondition: supplier bug. Assertions document the agreement and can check it at runtime.

Preserve promises

Subclasses must not strengthen preconditions or weaken postconditions. Inherited invariants still apply: demand no more, promise no less.

Fail honestly

Restore a consistent state; retry if a viable strategy exists, or report failure. Do not silently return as though the contract was fulfilled.

Main finding: replace redundant defensive checks with clear obligations.

Contracts support reliability; runtime checks alone do not prove correctness.

Meyer, "Applying 'Design by Contract'," Computer, October 1992, pp. 40–51.